Procesamiento de datos en streaming y batch con arquitecturas basadas en flujos de eventos

Gary Gianpier Choque Borda | Arthur Davis Huayaconza Condori

Directora: Margalida Salom Ramis

Trabajo Final de Máster

Master Universitario en Análisis y Visualización de Datos Masivos
Visual Analytics and Big Data – UNIR | Enero 2026

Índice de contenidos

1. Motivación y Contexto
2. Estado del Arte
3. El Modelo Dataflow
4. Objetivos y Metodología
5. Caso de Uso: COVID-19 Perú
6. Arquitectura del Sistema
7. Extensibilidad del Sistema
8. Dashboard y Visualizaciones
9. Resultados y Evaluación
10. Conclusiones
11. Referencias y Cierre

El desafío: batch vs. streaming

Procesamiento batch

- Datos históricos acumulados
- Alta latencia (horas/días)
- Resultados correctos pero tardos

Procesamiento en streaming

- Datos en tiempo real
- Baja latencia (ms/s)
- Reto: datos tardíos y desordenados

La brecha

Las organizaciones necesitan **ambos modos** con una única lógica de negocio. Las arquitecturas clásicas los tratan como sistemas separados.

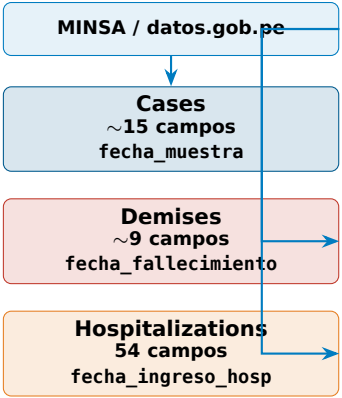
Contexto: vigilancia epidemiológica COVID-19

El caso real

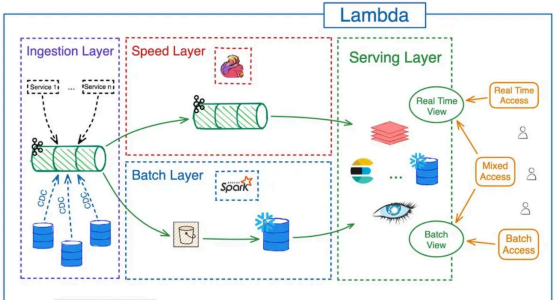
- Perú registró millones de casos COVID-19
- Datos públicos del MINSA en `datos.gob.pe`
- 39 archivos CSV con 3 schemas distintos
- Necesidad de vigilancia **en tiempo real** y análisis histórico **unificados**

Desafío de datos

- Datos tardíos: reportes con días de retraso
- Schemas heterogéneos (9–54 campos)
- Registros inválidos o incompletos



Estado del arte: Arquitectura Lambda



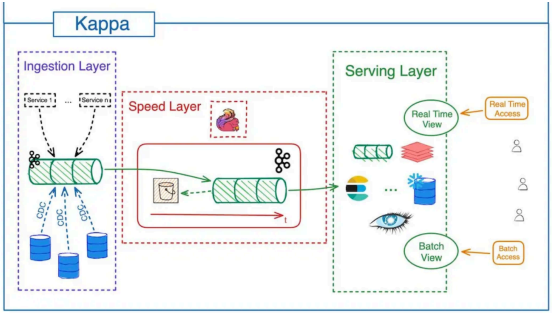
Lambda Architecture (Nathan Marz, 2011)

- **Batch layer**: reprocessa todo el historial
- **Speed layer**: procesa datos recientes
- **Serving layer**: combina ambas vistas

Limitaciones críticas

- Lógica de negocio **duplicada** en batch y speed
- Mantenimiento costoso y propenso a errores
- Inconsistencias entre capas

Estado del arte: Arquitectura Kappa



Kappa Architecture (Jay Kreps, 2014)

- **Una sola capa**: todo como streaming
- Lógica unificada en un pipeline
- Reprocesamiento mediante replay del log

Limitaciones

- Reprocesamiento masivo costoso en recursos
- Manejo de datos tardíos limitado
- Abstracción insuficiente para event time

Comparativa: Lambda vs. Kappa vs. Dataflow

Criterio	Lambda	Kappa	Dataflow
Lógica unificada	No	Sí	**Sí**
Datos tardíos (late data)	Parcial	Parcial	**Nativo**
Event time vs. processing time	Processing	Parcial	**Event time**
Complejidad operacional	Alta	Media	**Media**
Reutilización de código	No	Sí	**Sí**
Abstracción batch/stream	No	No	**PCollections**
Control de ventanas	Limitado	Parcial	**Nativo**

El modelo **Dataflow** (Akidau et al., 2015) unifica semantícamente batch y streaming bajo una única abstracción, superando las limitaciones de sus predecesores.

Modelo Dataflow: PCollections y Apache Beam

PCollections – la abstracción clave

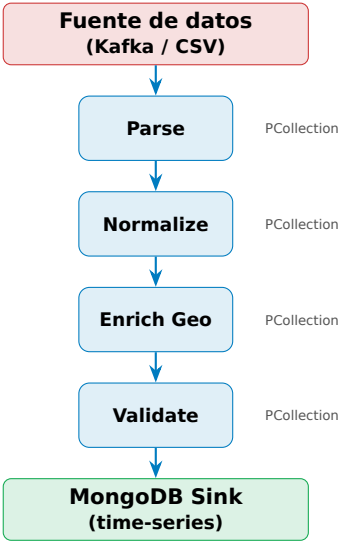
- Conjunto de datos **inmutable y potencialmente ilimitado**
- Mismo tipo representa batch *y* stream
- Transformaciones mediante `PTransform`
- El runner decide cómo ejecutar

Apache Beam

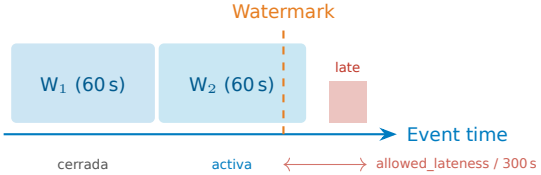
SDK open-source que implementa el modelo Dataflow.

Runners: Direct, Flink, Spark, Dataflow (GCP)

En este TFM: **DirectRunner** (local, reproducible)



Modelo Dataflow: Event Time, Watermarks y Windows



Event time vs. Processing time

- **Event time**: cuándo ocurrió el evento real
- **Processing time**: cuándo llega al sistema
- El Dataflow model trabaja sobre **event time**
- Garantiza resultados correctos incluso con retrasos

Configuración en este TFM

Ventanas fijas: **60 s**
Allowed lateness: **300 s** (5 min)
Trigger: `AfterWatermark` + `AfterProcessingTime`

Modelo Dataflow: Triggers

¿Qué es un trigger?

Controla **cuándo** se emiten los resultados de una ventana. Permite ver resultados parciales antes de que la ventana cierre.

Tipos de trigger principales

Trigger	Cuándo emite
AfterWatermark	Al cruzar el watermark
AfterProcessingTime	Tras N segundos
AfterCount	Al acumular N elementos
Repeat	Repite cualquier trigger

Configuración en el TFM

```
AfterWatermark
  .past_end_of_window()
 .with_early_firings(
  AfterProcessingTime
   .of(30)
 )
 .with_late_firings(
  AfterCount(1)
 )
```

Resultado

Emisión **temprana** cada 30 s (vista preliminar), emisión **final** al cerrar ventana, emisión adicional por cada dato tardío recibido.

Objetivos del TFM

Objetivo general

Diseñar e implementar un sistema de procesamiento de datos epidemiológicos que unifique batch y streaming mediante el **modelo Dataflow** sobre datos COVID-19 reales del MINSA Perú.

OE1 – Arquitectura Dataflow

Implementar pipeline Apache Beam con PCollections, watermarks, windows y triggers.

OE2 – Broker de mensajes

Integrar Apache Kafka como capa de desacoplamiento entre ingesta y procesamiento.

OE4 – Extensibilidad de schemas

Diseñar arquitectura modular para añadir nuevos schemas con mínimo esfuerzo.

OE5 – Dashboard en tiempo real

Visualizaciones interactivas con D3.js y Leaflet.js actualizadas via WebSockets.

OE6 – Evaluación y validación

Medir latencia, throughput y coherencia en 4 escenarios de prueba distintos.

6/6 objetivos específicos completados al 100 %

Metodología: CRISP-DM + Kanban

CRISP-DM adaptado

- **Business Understanding**: vigilancia epidemiológica
- **Data Understanding**: 3 schemas MINSA, 39 CSVs
- **Data Preparation**: normalización, enriquecimiento geo
- **Modeling**: pipeline Beam + ventanas temporales
- **Evaluation**: 4 escenarios de prueba
- **Deployment**: Docker Compose reproducible

Gestión con Kanban

- Tablero visual con columnas: Backlog $\rightarrow$ En progreso $\rightarrow$ Hecho
- Iteraciones cortas de 1–2 semanas
- Priorización continua de tareas
- Colaboración distribuida (2 autores)

Herramientas

GitHub + Docker + Python + XeLaTeX
Testing manual en 4 escenarios documentados

Caso de uso: Datos COVID-19 MINSA Perú

Fuente de datos

- **Origen**: datos.gob.pe (MINSA Perú)
- Datos reales anonimizados
- **39 archivos CSV** con millones de registros
- Tres schemas con estructuras heterogéneas
- Lectura eficiente con **Polars**

Schema	Campos	Timestamp
Cases	∼15	`fecha_muestra`
Demises	∼9	`fecha_fallecimie`
Hospitalizations	54	`fecha_ingreso_h`

Retos de los datos

- Valores nulos y formatos inconsistentes
- UBIGEO (código geográfico peruano) sin coordenadas
- Fechas en formato `YYYYMMDD` no estándar

Enriquecimiento geográfico

Cada registro incluye código UBIGEO.
El pipeline enriquece con: latitud, longitud, departamento, provincia, distrito – habilitando los mapas de calor.

Detalle de schemas: campos principales

(∼15 campos)

UUID, DEPARTAMENTO
PROVINCIA, DISTRITO
METODODX (tipo de prueba)
EDAD, SEXO
FECHA_RESULTADO
UBIGEO, id_persona

Timestamp: fecha_muestra

Demises (∼9 campos)

UUID, EDAD_DECLARADA
SEXO, DEPARTAMENTO
PROVINCIA, DISTRITO
UBIGEO

Timestamp: fecha_fallecimiento

Hospitalizations (54 campos)

id_persona, UBIGEO
SEXO, EDAD
UCI (booleano)
VENTILACION (booleano)
VACUNA1, VACUNA2, VACUNA3
flag_uci, n_dias_hosp
... (+44 campos más)

Timestamp: fecha_ingreso_hosp

Kafka Topics

topic: cases | **topic:** demises | **topic:** hospitalizations

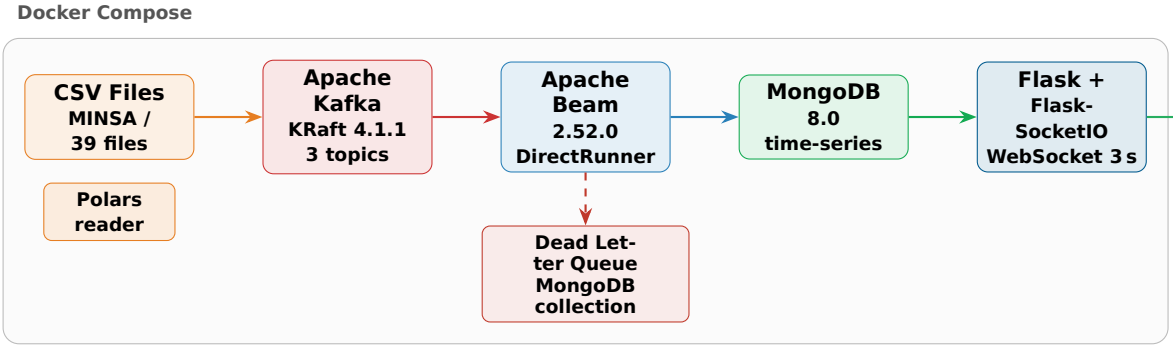
Cada topic corresponde a un schema y alimenta su propio pipeline paralelo.

Flujo de datos de extremo a extremo

Flujo de Datos End-to-End

Reemplazar con: flujo-datos-completo.png

Arquitectura general del sistema



Infraestructura: Docker Compose

Servicios en contenedores

- **kafka**: broker KRaft (sin Zookeeper)
- **init-kafka**: crea topics al inicio
- **mongodb**: base de datos time-series
- **pipeline-cases**: pipeline + ingesta cases
- **pipeline-demises**: pipeline + ingesta demises
- **pipeline-hospitalizations**: pipeline + ingesta hosp.
- **dashboard**: Flask + WebSockets + D3.js

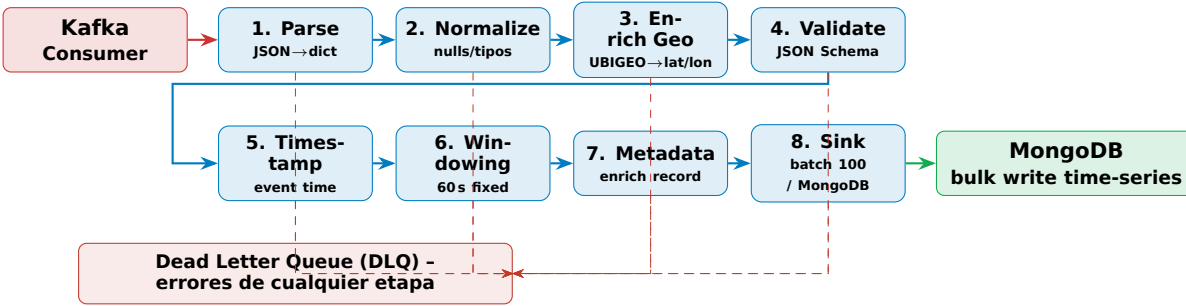
Ventajas del enfoque

- Entorno **completamente reproducible**
- Un solo comando: `docker compose up`
- Red interna entre servicios
- Fácil incorporación de nuevos pipelines

Paralelismo

Los 3 schemas procesan **en paralelo** en contenedores independientes, cada uno con su propio topic Kafka, instancia de pipeline Beam y colección MongoDB.

Pipeline Apache Beam: 8 etapas



Etapas 1–4: Preparación

Parse: string JSON a diccionario Python. **Normalize**: nulos a None, conversiones de tipo. **Enrich Geo**: código UBIGEO a lat/lon. **Validate**: validación contra JSON Schema.

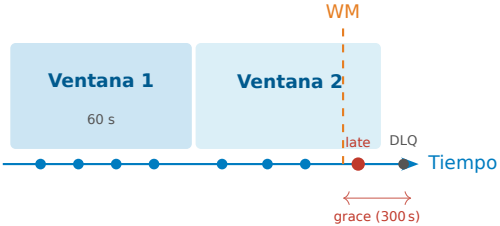
Etapas 5–8: Procesamiento temporal

Timestamp: campo fecha a event time. **Windowing**: ventana fija 60 s. **Metadata**: añade `pipeline_version`, `source_type`, `processed_at`, info de ventana. **Sink**: batch de 100, bulk write.

Ventanas temporales y manejo de datos tardíos

Configuración de windowing

- **Tipo**: FixedWindows (60 segundos)
- **Allowed lateness**: 300 s (5 minutos)
- Datos que llegan dentro de la ventana: procesados
- Datos tardíos dentro del grace period: **re-agregados**
- Datos más allá del grace: → **DLQ**



Metadata de ventana

Cada registro guardado incluye:
`window_start`, `window_end`, `pipeline_version`, `source_type`, `processed_at`

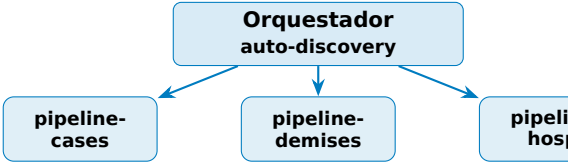
Dead Letter Queue y Orquestador de schemas

Dead Letter Queue (DLQ)

- Captura errores de **todas** las etapas del pipeline
- Colección `dead_letter_queue` en MongoDB
- Campos almacenados:
  - `error`: mensaje de error
  - `error_type`: tipo/etapa del fallo
  - `timestamp`: momento del error
  - `schema`: cases / demises / hosp
  - `record`: registro original completo
- Datos inválidos **no bloquean** el pipeline
- Permite auditoría y depuración post-facto

Orquestador automático

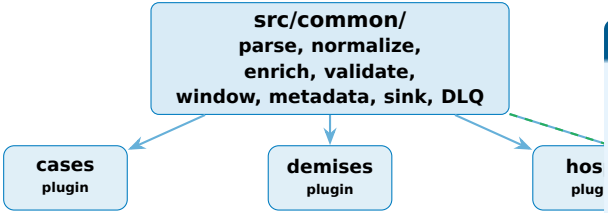
- Lee la carpeta `pipelines/`
- **Descubre schemas automáticamente**
- Lanza cada pipeline en su propio proceso
- No requiere configuración manual al añadir schemas



Diseño modular: extensibilidad de schemas

Principio de diseño

Toda la lógica compartida reside en `src/common/`. Cada schema es un **plugin** que solo configura su nombre, topic y colección.



Estructura de un nuevo schema

```
pipelines/vaccines/
  config.yaml
  schema.json
  pipeline.py
  ingestion.py
```

Solo cambia

- Nombre de clase en `pipeline.py`
- Topic Kafka en `config.yaml`
- Nombre de colección MongoDB
- Definición de campos en `schema.json`

Extensibilidad: añadir schema “vaccines” en 4 archivos

config.yaml

```yaml
schema_name: vaccines
kafka_topic: vaccines
mongodb_collection: vaccines
timestamp_field: fecha_vacunacion
window_size: 60
allowed_lateness: 300
```

pipeline.py (fragmento)

```python
class VaccinesPipeline(BasePipeline):
  schema = "vaccines"
  config = load_config(
    "pipelines/vaccines/"
  )
```

Lo que **NO** hay que escribir

- Lógica de parsing (ya en common)
- Normalización y validación (ya en common)
- Enriquecimiento geográfico (ya en common)
- Windowing y triggers (ya en common)
- DLQ handler (ya en common)
- Sink MongoDB (ya en common)
- Registro en el orquestador (auto-discovery)

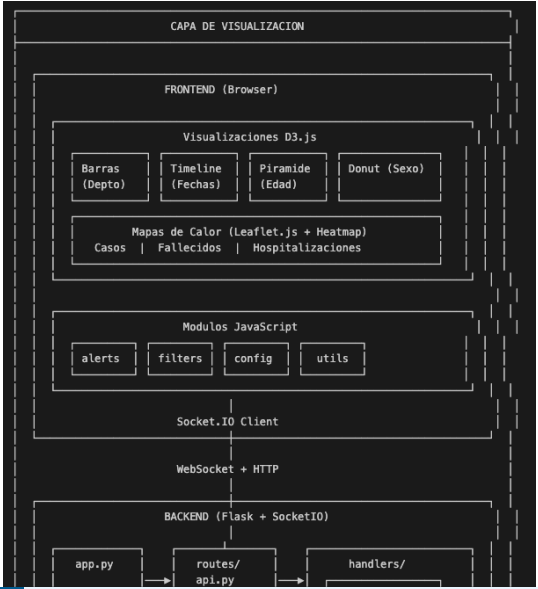
Resultado

Añadir un schema nuevo requiere ∼**50 líneas de código** frente a cientos si la lógica estuviera

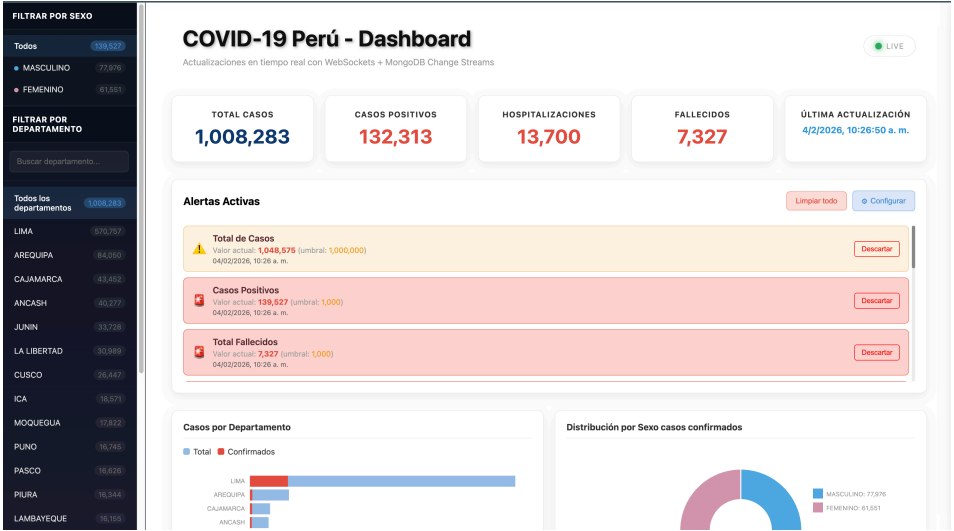
Dashboard: stack tecnológico y arquitectura

Backend (Flask)

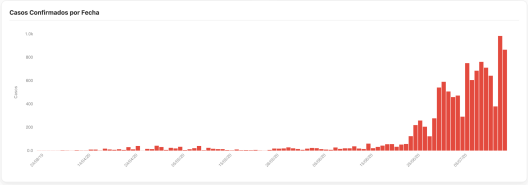
- **Flask** + **Flask-SocketIO**
- Emite actualizaciones vía WebSocket cada **3 s**
- Consulta MongoDB time-series en cada ciclo
- Agrega datos para cada una de las 9 visualizaciones
- Endpoints REST para datos históricos



Dashboard: vista general



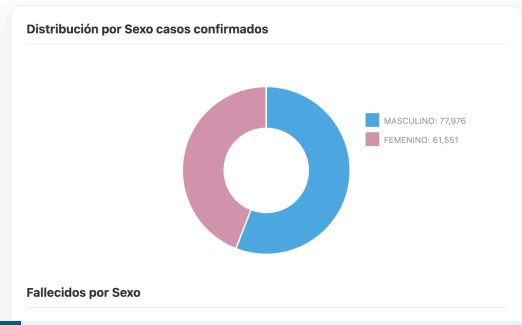
Visualizaciones: KPIs y gráficas analíticas

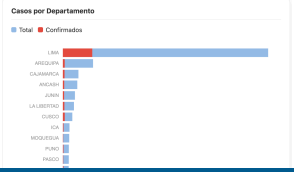


Serie temporal: casos/fallecidos por fecha

KPIs en tiempo real

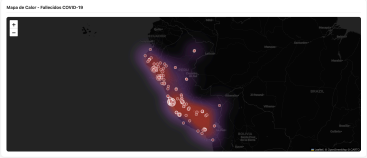
- Total casos confirmados
- Total hospitalizaciones activas
- Total fallecidos acumulados
- Actualización automática cada 3 s





Visualizaciones: mapas de calor georreferenciados

Casos COVID-19



Fallecidos

Hospitalizaciones

Implementación técnica

Leaflet.js + plugin **leaflet-heat** | Coordenadas lat/lon a partir del enriquecimiento UBIGEO en el pipeline | Intensidad proporcional al volumen por ubicación | Actualización en tiempo real via WebSocket

Resultados: 4 escenarios de evaluación

Escenario	Normal	Alta carga	Late data	Batch
Vol. mensajes/s	~100	~500	~100	500+
Latencia p50	baja	baja	baja	baja
Latencia p99	baja	baja	baja	moderada
Datos tardíos	n/a	n/a	procesados	n/a
DLQ rate	$< 1\%$	$< 2\%$	$< 1\%$	$< 1\%$
Coherencia	alta	alta	alta	alta

Throughput

Throughput escala **linealmente** con el volumen de entrada. Los 3 schemas procesan en paralelo sin degradación mutua.

Coherencia batch/stream

Los resultados de ambos modos (streaming y batch replay) producen **alta coherencia** en los agregados finales, validando el modelo Dataflow.

Validación de objetivos específicos

OE	Descripción	Estado	Evidencia
OE1	Pipeline Beam con PCollections, watermarks, windows	**100 %**	8 etapas
OE2	Kafka como capa de desacoplamiento (3 topics)	**100 %**	Docker logs
OE3	MongoDB time-series (4 colecciones)	**100 %**	Colecciones
OE4	Extensibilidad: nuevo schema en 4 archivos	**100 %**	Demo vaccines
OE5	Dashboard 9 visualizaciones + WebSocket	**100 %**	Screenshots
OE6	Evaluación en 4 escenarios de prueba	**100 %**	Tablas métricas

6 de 6 objetivos específicos completados al 100 %

Contribuciones técnicas del TFM

Implementación del modelo Dataflow

Primera implementación académica del modelo Dataflow (Akidau, 2015) aplicada a datos epidemiológicos reales del sistema de salud peruano.

Arquitectura extensible

Patrón “schema-as-plugin” con auto-discovery. Nuevos dominios de datos se integran con mínimo código adicional.

Validación empírica

Demostración de alta coherencia entre modo streaming y batch replay en datos epidemiológicos reales, validando la promesa central del modelo Dataflow.

Entorno reproducible

Sistema completo desplegable con un único comando Docker Compose. Facilita la reproducción y extensión del trabajo en investigaciones futuras.

Conclusiones

El modelo Dataflow funciona

Apache Beam demuestra que es posible unificar la lógica de batch y streaming en una sola abstracción (PCollections), eliminando la duplicación de Lambda y superando las limitaciones de Kappa para datos tardíos.

Extensibilidad sin coste

El patrón schema-as-plugin demuestra que es posible escalar a nuevos dominios de datos con mínimo esfuerzo, manteniendo toda la robustez del pipeline original.

Event time es clave

Trabajar sobre event time – en lugar de processing time – garantiza resultados correctos en presencia de retrasos y reordenamientos, crítico en vigilancia epidemiológica.

Impacto práctico

El sistema procesa datos reales del MINSA Perú, genera visualizaciones georreferenciadas en tiempo real y mantiene alta coherencia entre resultados, siendo aplicable a vigilancia epidemiológica real.

Limitaciones del trabajo

Limitaciones técnicas

- **DirectRunner**: entorno local, no distribuido. No valida comportamiento en clústers Flink/Spark.
- **Escala**: pruebas en máquina única, sin benchmark en entornos cloud.
- **Kafka sin replicación**: modo desarrollo, sin tolerancia a fallos de broker.
- **MongoDB local**: sin sharding ni replica set para alta disponibilidad.

Limitaciones de datos

- Datos anonimizados: no permiten trazabilidad individual completa.
- Cobertura UBIGEO: no todos los registros tienen código UBIGEO válido.
- Datos históricos: la ingesta simula streaming reproduciendo CSVs, no es producción real.

Alcance académico

El trabajo demuestra viabilidad técnica y valida el modelo Dataflow. No constituye un sistema de producción desplegado en el MINSA.

Líneas de trabajo futuro

Escalabilidad

- Migrar a **FlinkRunner** o **SparkRunner**
- Kafka en modo clúster con replicación
- MongoDB Atlas con sharding horizontal
- Benchmark en entorno cloud (GCP/AWS)

Machine Learning

- Detección de anomalías en streaming
- Pronóstico de picos epidemiológicos
- Modelos de clasificación de riesgo
- Integración en el pipeline Beam (MLTransform)

Nuevos schemas y dominios

- Schema `vaccines` (vacunación COVID)
- Datos de otras enfermedades (dengue, tuberculosis)
- Integración con datos del INEI (censo)

Operacionalización

- CI/CD para actualización de schemas
- Observabilidad: trazas, métricas, alertas (OTel)
- Despliegue en producción (MINSA /

Referencias principales

Artículos fundacionales

- **Akidau, T. et al.** (2015). “The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing.” *PVLDB*, 8(12).
- **Marz, N.** (2011). “How to beat the CAP theorem.” Lambda Architecture.
- **Kreps, J.** (2014). “Questioning the Lambda Architecture.” O’Reilly.
- **Akidau, T., Chernyak, S. & Lax, R.** (2018). *Streaming Systems*. O’Reilly Media.

Tecnologías

- Apache Beam 2.52.0 – *beam.apache.org*
- Apache Kafka 4.1.1 – *kafka.apache.org*
- MongoDB 8.0 – *mongodb.com*
- Flask-SocketIO – *flask-socketio.readthedocs.io*
- D3.js – *d3js.org*
- Leaflet.js – *leafletjs.com*
- Polars – *pola.rs*

Datos

- MINSA Perú. Datos COVID-19 anonimizados. *datos.gob.pe*
- Diccionario UBIGEO – INEI Perú

Gracias por su atención

Procesamiento de datos en streaming y batch
con arquitecturas basadas en flujos de eventos

Arthur Davis Huayaconza Condori

Directora: Margalida Salom Ramis

UNIR – Enero 2026

¿Preguntas?

Respaldo: arquitectura de módulos del sistema

```
visualization/
├── app.py                          # Servidor Flask + SocketIO + Polling
├── config.py                       # Configuracion (MongoDB, puertos, coords)
│
├── services/
│   ├── __init__.py
│   └── database.py                 # Cliente PyMongo
│
├── routes/
│   ├── __init__.py
│   └── api.py                      # 14 endpoints REST
│
├── handlers/
│   ├── __init__.py
│   ├── alerts.py                   # Sistema de alertas por umbral
│   ├── queries/
│   │   ├── __init__.py
│   │   ├── cases.py                # Queries: casos, heatmap, filtros
│   │   ├── demises.py              # Queries: fallecidos, heatmap
│   │   ├── hospitalizations.py     # Queries: hospitalizaciones
│   │   └── summary.py              # Queries: totales, conteos
│   └── websocket/
│       ├── __init__.py
│       └── events.py               # 15+ handlers WebSocket
│
├── static/
│   ├── css/
│   │   └── style.css               # Tema oscuro responsive
│   └── js/
│       ├── main.js                 # Entry point, conexion SocketIO
│       ├── charts/
│       │   ├── index.js            # Exporta todos los graficos
│       │   ├── department.js       # Barras horizontales (D3.js)
│       │   ├── timeline.js         # Area chart temporal (D3.js)
│       │   ├── age.js              # Piramide poblacional (D3.js)
│       │   ├── sex.js              # Donut chart (D3.js)
│       │   └── heatmaps.js         # Mapas Leaflet + leaflet.heat
│       └── modules/
│           ├── alerts.js           # UI alertas, notificaciones
│           ├── filters.js          # Filtros departamento/sexo
│           ├── config.js           # Constantes frontend
│           └── utils.js            # Formateo numeros, fechas
│
└── templates/
    └── index.html                  # SPA con todas las visualizaciones
```